Course Name:	**COMP 2233 Data Structures**		
Lecturers:	Assoc. Prof. Dr. Ömer ÇETİN Research Assistant Giray KILIÇ Research Asistant Hatice ÇAĞLAYANLAR		
Lab. Number:	2	**Lab. Date:**	29.09.2026 / 02.10.2026
Objectives:	The main objective of this laboratory exercise is to practically demonstrate to students the direct impact of data structure selection on algorithm design and system performance. Within the scope of this study, students enhance their algorithmic thinking skills by modeling problem solutions using pseudocode and flowcharts prior to implementation. By developing two distinct C implementations—one using discrete variables and the other using array data structures—students directly observe the code flexibility and memory efficiency gained through contiguous memory allocation and iterative loop structures. Finally, comparing these implementations via Linear Search and Binary Search algorithms solidifies their understanding of how selecting the right data structure and algorithm improves time ($Big-O$) and space complexity.		

Course Learning Outcomes (CLO):		**Relevancy (1-5)**
CLO (In1)	Students will be able to identify useful data structures to represent specific types of information.	4
CLO (Sk3)	Students will be able to design and implement linear data structures.	3
CLO (Co2)	Students will be able to design and implement efficient solutions for searching and sorting operations.	4

Pre-Lab Assignment

To ensure students enter the laboratory session fully equipped to complete the tasks efficiently within the allocated timeframe, the **Pre-Lab Assignment (Pre-Lab Work)** is defined below. Prior to attending the laboratory session, students are required to complete the following theoretical, practical, and design tasks:

1. Theoretical Preparation & Reading Topics

- **C Language Fundamentals:** Review the inner mechanics of strcmp() and strcpy() functions from the <string.h> library, paying attention to their return values (0, <0, >0) and how strings are represented as character arrays (char[]) in memory (including the role of the null terminator \0).
- **Data Structures:** Revisit single and multi-dimensional character arrays (char list[10][16]) as well as custom structure definitions (struct) in C.
- **Complexity Analysis:** Review the formal definition of $\boldsymbol{Big-O}$ notation and understand **Best Case**, **Worst Case**, and **Average Case** execution scenarios.

2. Algorithmic Design Preparation

- **Pseudocode Drafting:** Write initial drafts of the pseudocode for both experiments, ensuring logical step-by-step sequencing.
- **Flowchart Construction:** Graphically construct flowcharts mapping decision points (if-else), iterative loops (for/while), and start/end boundaries on paper.

3. Development Environment Setup

- Ensure a working C compiler (GCC, Clang, MSVC) and an IDE/editor (VS Code) are installed and verified on your environment.
- Study methods to prevent buffer overflow vulnerabilities when handling string inputs via scanf() or fgets().

Pre-Lab Self-Check Questions (Mental Readiness):

- *Check 1:* Why does strcmp("apple", "apple") return 0 instead of 1 (true)? How does this impact your if condition check inside a loop?
- *Check 2:* If you search for an item in a list of 10 elements using Linear Search, what is the maximum number of string comparisons required in the worst-case scenario?
- *Check 3:* Why can you NOT write a simple for loop to iterate through variables named item0, item1, item2... item9?

Laboratory Experiment

COMP 2233 / Data Structures Laboratory — Experiment 1

Topic: Algorithm Development, Flowcharts, Choice of Data Structures, and Algorithm Complexity ($Big - O$)

1. Problem Description

For a supermarket automation system, a shopping list containing N items needs to be searched to verify whether a specific item requested by the user is present in the list.

2. Constraints and Specifications

- N (List Length): Maximum 10 items ($N \leq 10$).
- Item Name Length: Each item name stored in the list can be at most 15 characters long (excluding the null terminator \0 in C).

3. Experimental Tasks and Deliverables

Complete the following steps sequentially:

1. Pseudocode: Design a clear and readable pseudocode to solve the problem.
2. Flowchart: Illustrate the step-by-step logic of the algorithm using a flowchart.
3. Implementation and Complexity Analysis Using Discrete Variables:
    - Assuming no array data structure is used, write a C program segment that performs a linear search across separate variables (item0, item1, ... item9).
    - Calculate the time complexity ($Big - O$) and space complexity of this approach.
4. Implementation and Complexity Analysis Using Array Data Structure:
    - Solve the problem in C using a string array structure (char list[10][16]).
    - Calculate the time complexity ($Big - O$) of this approach.
5. Comparison and Evaluation:
    - Compare and evaluate the two solutions from Steps 3 and 4 in terms of code flexibility, memory arrangement, and the performance of Linear Search ($O(N)$) versus Binary Search ($O(\log N)$) algorithms using $Big - O$ notation.

Sample Terminal I/O (Experiment 1):

```
Enter item to search: milk
Searching using discrete variables... Found at position 2.
Searching using array structure... Found at index 2.
Execution Time: 0.000012 seconds | Big-O Complexity: O(N)
```

COMP 2233 / Data Structures Laboratory — Experiment 2

Topic: Sequential Processing vs. Struct Arrays, Dynamic Search Mechanics, and $Big - O$ Complexity Analysis

1. Problem Description

An online flight ticketing system needs to search through a passenger manifest containing N flight bookings to verify whether a passenger with a specific **6-digit PNR Code** exists in the system and retrieve their seat status.

2. Constraints and Specifications

- N **(Manifest Capacity):** Maximum 10 passenger records ($N \leq 10$).
- **Data Fields per Passenger:**
  - **PNR Code:** Exactly 6 alphanumeric characters (e.g., "TK82F1" + null terminator \0).
  - **Passenger Name:** Up to 15 characters (e.g., "Merve Kaya" + null terminator \0).
  - **Seat Number:** Integer value between 1 and 180.

3. Experimental Tasks and Deliverables

Complete the following steps sequentially:

1. **Pseudocode:** Design a pseudocode that takes a target PNR code from the user, searches the manifest, and prints the passenger's name and seat number if found.
2. **Flowchart:** Construct a flowchart representing the decision-making process and loop termination conditions of your algorithm.
3. **Implementation and Complexity Analysis Using Discrete Variables:**
   - Assuming no composite data structures or arrays are allowed, write a C program segment using discrete variables for each passenger attribute (e.g., pnr0, name0, seat0, pnr1, name1, seat1, ...).
   - Perform a linear check across all individual variables to locate the target PNR.
   - Calculate the time ($Big - O$) and space complexity of this approach.
4. **Implementation and Complexity Analysis Using Array of Structures (struct):**
   - Define a Passenger structure type containing pnr, name, and seatNumber.
   - Create an array of structures (struct Passenger manifest[10]) and write a modular C program to perform the search using an iterative loop.
   - Calculate the time ($Big - O$) and space complexity of this array-based approach.
5. **Comparison and Evaluation:**
   - Evaluate both implementations regarding code maintainability, memory alignment, and cache locality.
   - Compare the efficiency of searching an unsorted manifest via **Linear Search** ($O(N)$) against a PNR-sorted manifest using **Binary Search** ($O(\log N)$).

Sample Terminal I/O (Experiment 1):

```
Enter 6-digit PNR Code: TK82F1
[SUCCESS] Passenger Found!
Name: Merve Kaya | Seat Number: 42
Search Method: Struct Array Loop | Time Complexity: O(N)
```

Additional Knowledge (Supplementary Reference)

Review the following core concepts to guide your analysis, complexity calculations, and comparisons in both experiments:

1. **Sequential vs. Random Memory Access ($O(1)$ Indexing)**
   - **Discrete Variables:** Declaring separate variables (e.g., item0, item1 or pnr0, pnr1) causes the compiler to allocate memory slots non-contiguously. Accessing the "i-th" element requires explicit if-else branching, making programmatic iteration impossible.
   - **Arrays / Struct Arrays:** Arrays allocate a single, continuous block of memory. The address of any element at index i is calculated instantly using the formula:

     $$\text{Address}(i) = \text{Base Address} + (i \times \text{Size of Element})$$

     This mathematical calculation grants $O(1)$ **random access** to any array index.

2. **CPU Cache Locality and Cache Line Prefetching**
   - **Spatial Locality:** Modern CPUs load data into fast cache memory (L1/L2/L3) in chunks called *Cache Lines* (typically 64 bytes).
   - When an array element is accessed, adjacent elements residing in the same continuous block are automatically preloaded into the CPU cache (**Cache Hit**).
   - Discrete variables scattered across memory frequently result in **Cache Misses**, forcing the CPU to retrieve data directly from slower main memory (RAM), which degrades runtime performance.
3. **Time and Space Complexity ($Big - O$ Notation)**
   - **Time Complexity ($Big - O$):** Measures how the execution time of an algorithm grows as the input size N scales to infinity.
     - **Best Case ($O(1)$):** The target element is located at the very first position.
     - **Worst Case ($O(N)$):** The target element is at the last position or not present at all, requiring N comparisons.
   - **Space Complexity ($Big - O$):** Measures the total memory allocated by the algorithm. Both discrete variables and array structures require $O(N)$ total memory, but arrays minimize overhead in symbol table management and instruction binary size.
4. **Linear Search vs. Binary Search Mechanics**
   - **Linear Search ($O(N)$):** Sequentially checks every element from index 0 to $N - 1$. It operates on **unsorted** data, making it versatile but inefficient for large datasets.
   - **Binary Search ($O(\log N)$):** Repeatedly divides the search space in half by comparing the target with the middle element.
   - **Prerequisite:** Binary Search **strictly requires a sorted dataset** (e.g., alphabetically or numerically). If the array is unsorted, sorting it first takes $O(N \log N)$ time, which may outweigh the benefits for a single search operation on small inputs like $N = 10$.
5. **String Comparisons in C (strcmp)**
   - In C, strings cannot be compared directly using equality operators (e.g., str1 == str2 checks memory addresses, not content).
   - You must use strcmp(str1, str2) from <string.h>, which compares characters ASCII value by ASCII value.
   - For strings of maximum length M, strcmp runs in $O(M)$ time in the worst case. Therefore, the complete search complexity across N elements is rigorously expressed as $O(N \cdot M)$. Since M is fixed ($M \leq 16$), it simplifies to $O(N)$.

Attachments
Attachment A: ANSI/ISO Flowchart Symbols Reference Guide (`flowchart_symbols.pdf`) Attachment B: Memory Layout and String Handling in C Cheat Sheet (`c_string_memory_guide.pdf`) Attachment C: Lab 2 C Code Starter Template (`lab2_template.c`)
References
1. Weiss, M. A. (2013). Data Structures and Algorithm Analysis in C (2nd ed.). Pearson. 2. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4th ed.). MIT Press. 3. Kernighan, B. W., & Ritchie, D. M. (1988). The C Programming Language (2nd ed.). Prentice Hall. 4. ISO/IEC 5807:1985. Information processing — Documentation symbols and conventions for data, program and system flowcharts.